

HANDS_ON_Erster_Flow

Hands-on: Einen einfachen Flow Schritt für Schritt bauen

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login `admin` `N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0` Fertiges Referenz-Lab auf der Canvas: **“Lab 0 - Einfacher Flow”** (zum Vergleich/Spicken)

Lernziel: Den kompletten NiFi-Grundzyklus einmal selbst durchlaufen: Quelle → Verbindung/Queue → Transformation → Ziel, mit bewusst verdrahteten Beziehungen, dann validieren, starten und das Verhalten beobachten.

Zielflow:

```
GenerateFlowFile —success—> ReplaceText —success—> PutFile
  (Quelle)                  (Transform)          (Ziel)
```

Wir erzeugen Text, ersetzen ein Wort, und schreiben das Ergebnis als Datei.

Schritt 1 : Quelle hinzufügen (GenerateFlowFile)

GenerateFlowFile erzeugt künstliche FlowFiles : ideal zum Testen ohne externe Datenquelle. Alternative: GetFile (liest echte Dateien aus einem Ordner, siehe andere Labs).

1. Prozessor auf die Canvas ziehen: oben in der Toolbar das **Processor-Icon** auf die Fläche ziehen (oder Rechtsklick → Add Processor).
2. Im Filter `GenerateFlowFile` eingeben → auswählen → **Add**.
3. Doppelklick → Tab **Properties**:
 - **Custom Text** = `Hallo NiFi, heute ist ${now():format('yyyy-MM-dd')} (${now())}` ist NiFi Expression Language : wird zur Laufzeit ausgewertet.)
 - **Data Format** = `Text`
 - **Batch Size** = `1`
4. Tab **Scheduling: Run Schedule** = `10 sec` (erzeugt alle 10s eine FlowFile, sonst flutet es).
5. **Apply**.

Hinweis: GenerateFlowFile hat nur die Relationship `success`. Es ist eine reine Quelle : kein Eingang.

Schritt 2 : Transformationsschritt ergänzen (ReplaceText)

1. Zweiten Prozessor hinzufügen → Filter `ReplaceText` → Add. Unter die Quelle platzieren.

2. Doppelklick → **Properties:**

- **Replacement Strategy** = `Literal Replace` (einfaches Suchen-und-Ersetzen ohne Regex)
- **Search Value** = `Hallo`
- **Replacement Value** = `Servus`
- **Evaluation Mode** = `Entire text`

3. **Apply.**

ReplaceText hat zwei Relationships: `success` (Ersetzung ok) und `failure` (z.B. FlowFile größer als Maximum Buffer Size). Beide müssen später verdrahtet ODER terminiert werden.

Schritt 3 : Verbindung mit Queue anlegen (Quelle → Transform)

Eine Verbindung IST die Queue. Sie transportiert FlowFiles und puffert sie, bis der nächste Prozessor sie abholt.

1. Maus über **GenerateFlowFile** bewegen → es erscheint ein **Kreis-Pfeil-Symbol** am Rand der Box.
 2. Dieses Symbol auf **ReplaceText** ziehen und loslassen.
 3. Im Dialog **Create Connection:** Relationship `success` ankreuzen → **Add**.
 4. Auf der Verbindung erscheint jetzt eine **Queue** (zeigt später "queued = N", FlowFile-Zähler).
-

Schritt 4 : Zielprozessor konfigurieren (PutFile)

1. Dritten Prozessor hinzufügen → Filter `PutFile` → Add. Unter ReplaceText platzieren.
 2. Doppelklick → **Properties:**
 - **Directory** = `/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/output/lab0`
 - **Create Missing Directories** = `true`
 - **Conflict Resolution Strategy** = `replace`
 3. **Apply.**
 4. Verbindung **ReplaceText** → **PutFile** anlegen (wie Schritt 3): Relationship `success` ankreuzen.
-

Schritt 5 : Beziehungen bewusst verdrahten (success / failure / unmatched)

Jede Relationship muss **entweder verbunden ODER terminiert** sein : sonst bleibt der Prozessor **invalid** (gelbes Warndreieck △) und startet nicht.

Prozessor	Relationship	Was tun	Warum
GenerateFlowFile	success	→ ReplaceText (verbunden)	normaler Weg

Prozessor	Relationship	Was tun	Warum
ReplaceText	success	→ PutFile (verbunden)	erfolgreich ersetzt
ReplaceText	failure	terminieren	für die Demo; in Produktion → LogAttribute/PutFile
PutFile	success	terminieren	Endknoten, FlowFile fertig
PutFile	failure	terminieren	Endknoten (Schreibfehler)

Relationship terminieren: Prozessor doppelklicken → Tab **Relationships** → bei der jeweiligen Relationship das Häkchen **terminate** setzen → Apply.

`unmatched` taucht bei diesem Flow nicht auf : sie gehört z.B. zu `RouteOnAttribute/RouteOnContent` (FlowFiles, die auf keine Regel passen). Merksatz gilt aber identisch: jede Relationship verbinden oder terminieren. (Beispiel mit `unmatched`: Lab F2 / Lab 4.)

Schritt 6 : Validieren, starten, beobachten

Validieren

- Jeder Prozessor zeigt links oben ein Symbol:
 -  gelb = **invalid** (Maus drüber zeigt den Grund, meist "Relationship X not connected/terminated")
 -  rot/grau = **gestoppt, aber valide** (startklar)
- Erst weiter, wenn alle drei das Stopp-Symbol (kein ) zeigen.

Starten

- Einzel: Prozessor markieren →  in der Toolbar (oder Rechtsklick → Start).
- Alle zusammen: leere Canvas-Stelle klicken (alles abwählen) → Rechtsklick → **Start**, oder die Process Group starten.

Beobachten (das Wichtige)

1. **Queue-Zähler** auf den Verbindungen: steigt/fällt, während FlowFiles fließen.
2. **Prozessor-Statistik** (In/Read-Write/Out) : die Zahlen sind das **5-Minuten-Fenster**.
3. **Ergebnis prüfen:**

```
ls -l $N/data/output/lab0/
cat $N/data/output/lab0/*           # -> "Servus NiFi, heute ist 2026-..."
```

Aus `Hallo` wurde `Servus` : die Transformation hat gegriffen.

4. **Data Provenance:** Rechtsklick auf ReplaceText → **View data provenance** → Event öffnen → Tab **Content** zeigt Inhalt **vorher** (Hallo...) und **nachher** (Servus...). Das macht die Transformation sichtbar.

Stoppen & Reset

```
# In der UI: Process Group / Prozessoren stoppen (■)  
find $N/data/output/lab0 -type f -delete
```

Achtung: GenerateFlowFile produziert alle 10s weiter, solange es läuft : nach der Beobachtung **stoppen**, sonst füllt sich `output/lab0` endlos.

Verifiziertes Ergebnis (im Live-Test bestätigt)

- Eingabe (GenerateFlowFile): `Hallo NiFi, heute ist 2026-06-02`
- Nach ReplaceText / in der Datei: `Servus NiFi, heute ist 2026-06-02`
- Datei landet in `data/output/lab0/` mit eindeutigem Namen (UUID).

Häufige Stolpersteine

- **Prozessor bleibt**  **invalid** → eine Relationship ist weder verbunden noch terminiert (Tab Relationships).
- **Nichts im Output** → läuft GenerateFlowFile? (▶). Bei GetFile zusätzlich an das 30s-Polling denken.
- **Output flutet** → GenerateFlowFile Run Schedule zu klein; `10 sec` setzen und nach dem Test stoppen.
- `${now()}` **erscheint wörtlich** statt Datum → Data Format muss `Text` sein und der Ausdruck korrekt geschrieben (`${now():format('yyyy-MM-dd')}`).